DD2459 Software Reliability 2024
Take-home Exam
Tuesday 5 March, 15.00 – Thursday 7 March, 12.00 midday
(Version 1.0)

Instructions.
Please read the following instructions **carefully**.

(1) Only hard-copy exam submissions are accepted. Clearly mark at the top of each sheet of paper you use: (a) your name, (b) the page number.

(2) On your front page indicate: (a) how many pages are contained in your work in total, (b) your name (c) your personal or KTH number, (d) your e-mail address (in case I need to contact you)

(3) There are two ways to **submit** your work. (**a**) Your work may be **handed in personally by you only** to EECS studentexpedition, E building, level 4, Lindstedsvägen 3, no later than Thursday 7th March 2024 at 12.00 midday. After this time, it will be marked as late, and marks will be subtracted. **Please take identification with you if you submit this way, e.g. a legitimation or passport.** (**b**) If you are unable to reach the studentexpedition yourself (for example if you are at work) you may **post** your manuscript to: Karl Meinke, EECS School, KTH Kungliga Tekniska Högskolan, 100 44 Stockholm, Sweden. The date on the postmark will be taken as the date of your submission. The deadline of March 7th, 2024 applies to all manuscripts submitted by post.

(4) If manuscripts are submitted in any other place or by any other means than those described in Part (3) then the examiner and EECS School cannot be held responsible in the case that manuscripts are lost. In the case of postal submission, it is strongly recommended that you keep a digital copy or photocopy in case of postal loss. KTH cannot be held responsible for loss in any national postal system.

(5) If you have any questions about the exam (for example, if you do not understand a question) you may call the examiner on 08 790 6337. Please do not call before 10.00 am or after 5.00pm, and be aware that I may be in a meeting! You can also e-mail the examiner at karlm@kth.se.

(6) I will publish any typographical errors and corrections that become known during the exam on the course exam web page and notify you with Canvas. Therefore, you should check back regularly on the course exam page for updates that will have new version numbers 1.x.

(7) You may use your course notes, books and the internet. However, all material you submit must be your own. (a) You are not allowed to discuss your answers with anyone else until the exam is finished. (b) You are not allowed to copy anyone else's work. (c) *By handing in your manuscript you are declaring that you have read and abide by all the rules on this cover page*. (d) In the case that cheating is suspected, disciplinary action will be taken.

(8) Hand-written answers must be written clearly. No marks will be awarded for work that I cannot read. You can write your answers in Swedish or in English. You can submit a hand-written manuscript or a typed manuscript or any combination of both.

NOTE: For a full score of 100 points you should answer <u>all 4 questions below</u>.

Question 1. (Total 35 points)

Write appropriate *pre-* and *post-conditions* for the following mathematical operations using the JML specification language (i.e. write *requires-ensures* conditions).
You should try to avoid both over and under-specification, i.e. write all and only the constraints necessary for each pre- and post condition. Do not omit relevant constraints or add irrelevant ones.
If you are unsure how any mathematical operation/method is defined you may look it up online or in a book.
You <u>do not</u> have to provide Java code for any of the operations, <u>only JML</u>.

Recall that a *finite directed graph* G = (V, E $\subseteq$ V $\times$ V) over a node set V = $\{v_1, \ldots, v_n\}$ can be represented on a computer by a 2-dimensional square n$\times$ n Boolean array that encodes the *adjacency matrix*

`boolean [] [] Adj`

where

for all 1 <= i, j <= n : `Adj[i][j] == true` if, and only if (v_i, v_j) $\in$ E.

A *clique* C in G is a subset C $\subseteq$ V such any two distinct nodes v, w $\in$ C are connected by an edge (v, w) $\in$ E.

(i) **(10 points)** The *clique property on an <u>undirected</u> graph G*

`boolean clique(boolean [][] Adj, int [] nList)`

Input: An adjacency matrix `Adj` for an undirected graph `G` and a list `nodeList` of nodes occurring in G.
Output: `true` if and only if the set of nodes occurring in `nodeList` forms a clique in G.

Model Answer
Note, here either:
(a) we assume `nList` is a list without repetitions (i.e. a <u>set</u> representation of the clique, which simplifies the postcondition) or
(b) we do not (i.e. a <u>list</u> representation of the clique which complicates the postcondition somewhat).
It doesn't matter which assumption we choose as long as we are consistent and explicit throughout. Below I will allow repetitions in `nList`, as the question doesn't indicate otherwise.

```
 requires

// Adj is a well defined square matrix
(
 Adj != null &
```

```
      (\forall int i; 0 <= i < Adj.length;
      Adj[i] != null &
      Adj[i].length == Adj.length)
    )

    &
    // nList is a well defined array
    nList != null &

    &
    // G is undirected i.e. symmetric
    (
      (\forall int i; 0 <= i < Adj.length;
      (\forall int j; 0 <= j < Adj.length;  Adj[i][j] <=>
      Adj[j][i])
    )

    &
    // every element of nList is a node in G
    // Notice that repetitions in nList don't matter here!!!
    (\forall int i; 0 <= i < nList.length; 1 <= nList[i] <=
    Adj.length)


ensures
// \result indicates a genuine clique

\result == true <=>

(\forall int i; 0 <= i < nList.length;
       (\forall int j; 0 <= j < nList.length;
              nList [i] <> nList[j] =>
              Adj[nList[i]][ nList[j] ] == true)
)
```

(ii) **(10 points)** A *maximal clique in an undirected graph G*

`int [] maximalClique(boolean [][] Adj)`

Input: An adjacency matrix `Adj` for an undirected graph G.
Output: A maximal sized clique in G (represented as a nodelist), i.e. a clique that is not contained in any larger clique in G.

You should reuse your answer to Part (i) to make the JML in your answer shorter.

Model Answer

```
    requires
```

```
      // Adj is a well-defined square matrix
      (
       Adj != null &
       (\forall int i; 0 <= i < Adj.length;
       Adj[i] != null &
       Adj[i].length == Adj.length)
      )

      &
      // G is undirected i.e. symmetric
      (
       (\forall int i; 0 <= i < Adj.length;
       (\forall int j; 0 <= j < Adj.length;
              Adj[nList [i]][j] <=> Adj[nList [j]])
      )



Ensures

      // \result is well defined
      \result != null

      &
      // every element of \result is a node in G
      (\forall int i; 0 <= i < \result.length; 1 <= \result [i]
      <= Adj.length)

      &
      // \result is a clique
      clique(Adj, \result)

      &

      // \result is a maximal clique,
      // i.e. if c is any larger clique containing \result
      // then c must have repetitions.

      // If there exists a larger list c
      (\exists int[] c; c.length > \result.length;
      // such that c is a clique
      clique(Adj, c) &
      // and c contains \result
      (\forall int i; 0 <= i < \result.length;
            (\exists int j; 0 <= j <= c.length; \result[i] ==
      c[j])
      // then c contains repetitions
      =>
            (\exists int i; 0 <= i <= c.length;
                  (\exists int j; 0 <= j <= c.length;
                        i != j & c[i] == c[j])
```

)
)
)
)
)

(iii) **(10 points)** Write out an algorithm (i.e. pseudo-code) for a test script which uses your JML answer to Part (i) to generate and execute one random test case `(A, C)` for the method

```
boolean clique(boolean [][] Adj, int [] nList)
```

consisting of an undirected adjacency matrix `A and` a nodeList `C` in `A`. Be careful to show clearly how the postcondition of Part (i) is translated into a test oracle to return a verdict in your script.

Finally, carefully analyse whether a randomly chosen test case `(A, C)` generated by your script is more likely to be a test of a valid clique or an invalid clique for the SUT `clique`.

Model Answer

An algorithm is as follows:

Choose a random graph size n > 1 and initialize a square n x n Boolean matrix A.
Choose a random clique size 1 < m < n and initialize a linear integer array C of length m.
Randomly assign a Boolean value b_{ij} to both A[i][j] and A[j][i] (so A is undirected) for all 0 <= i, j < n, e.g. setting A[i][j], A[j][i] := true with probability p.
Randomly choose the clique members C_1 , …, C_m in 0 ,…, n-1, e.g. with equi-probability p' = 1/n.

Execute SUT with
v := `clique(A, C)`

If $A[C_i][C_j]$ == true for all C_i, C_j then if v == true return pass
If $A[C_i][C_j]$ == false for some C_i, C_j then if v == false return pass
else return fail

The last two lines are based on the postcondition, while initializations reflect the three preconditions.

(6 points) There are 3 key observations regarding randomly generated test cases:

(1) We need to assume the proportion of edges in `A` is constant as n increases and less than $n^2/2$ total edges. Otherwise `(A, C)` is highly likely to be an invalid (respectively valid) clique as n increases.
(2) For a given but randomly chosen graph size n, a randomly chosen test case `(A, C)` is more likely to represent an invalid clique as the size m of the nodeList `C` increases. This problem increases as n increases.
(3) Large nodeList sizes are just as likely as small sizes if the nodeList size m is equiprobably distributed in the range 1 ,…, n.

So the overall bias is towards invalid cliques as random test cases.

(iv) **(5 points)** Suppose you implement `boolean clique(boolean [][] Adj, int [] nList)` and it successfully passes your test script in Part (iii). Is it safe to use your implementation on an adjacency matrix `Adj` for a general directed graph that may not be undirected? Explain your answer.

Model Answer
There are two points to cover here.

(1) If G is a directed graph that is not undirected, then the precondition of (i) is not satisfied. Therefore, the postcondition of (i) is not technically applicable, by the "partial correctness interpretation" and "all bets are off". We are not actually guaranteed anything about the output.
(2) Suppose that the postcondition of (1) is valid anyway (a plausible assumption, since the postcondition ought to correspond to some implementation code). Then the result corresponds to testing just one of each symmetric pair of edges (v,w) and (w,v) in G. If we choose the wrong one (under the now false assumption of symmetry) we can arrive at either a false positive or a false negative.

To eliminate this problem the postcondition of part (i) could be strengthened to:

```
\result == true <=>

(\forall int i; 0 <= i < nList.length;
(\forall int j; 0 <= j < nList.length;
       Adj[i][j] == true | Adj[j][i] == true
)
```

And this still works for the special symmetric case. Notice the additional disjunct `Adj[j][i] == true`. Notice that an implementation might now require some extra coding,

Question 2. (Total 10 points)

(i) **(5 points)** Recall that sets of integers can be represented as lists of integers. Write down three different metamorphic equations that should be satisfied by a test for equality on integer sets:

```
boolean SetEquality(List<int> firstSet, List<int>
secondSet)
```

with

Input: Two lists `firstSet, secondSet` of `int`.
Output: `true` if the lists `firstSet` and `secondSet` are equal as sets otherwise `false`.

Clearly define the data transformation

T: `List<int> * List<int>` $\rightarrow$ `List<int> * List<int>`

used on the input pair for each of your three different equations.

Model Answer

Here are five metamorphic equations that are possible answers. Can you find more?

(1) Commutativity: `SetEquality`(l_1, l_2) = `SetEquality`($T_1(l_1,l_2)$) where $T_1(l_1,l_2)) = (l_2,l_1)$

(2) Weak Order Independence: `SetEquality`(l_1, l_2) = `SetEquality`($T_2(l_1,l_2)$) where $_2(l_1,l_2)) = (l_1$,reverse(l_2))

(3) Strong Order Independence: `SetEquality`(l_1, l_2) = `SetEquality`($T_3(l_1,l_2)$) where $T_3(l_1,l_2)) = (l_1$,permute(l_2)) and permute is any fixed list permutation

(4) Repetition Independence: `SetEquality`(l_1, l_2) = `SetEquality`($T_4(l_1,l_2)$) where $T_4(l_1,l_2)) = (append(l_1$, c),append(append(l_2,c),c)) where c is any integer

(5) Substitution law: `SetEquality`(l_1, l_2) = `SetEquality`($T_5(l_1,l_2)$)
Where T_5 is an elementwise transformation
$T_5(l_1,l_2)) =(f(l_1), f(l_1))$ and
$f((x_1, …, x_n)) = (f(x_1), …, f(x_n))$ for any <u>injective</u> unary integer operation f : int -> int, e.g. f(x) = x+1.
Here injectivity is necessary, otherwise unequal sets may become equal after transformation by T_5.

Notice that adding the same element x to both lists l_1,l_2 does <u>not</u> preserve inequality. For example, if x is initially in l_1 but not in l_2. So, this transformation does <u>not</u> give a metamorphic equation.

(ii) **(5 points)** For each of your three metamorphic equations e_i , i = 1, 2, 3, derived for Part (i), consider its associated transformation T_i. Starting from one seed test case

(x_0,y_0) : `List<int> * List<int>`

discuss whether the sequence of test cases

$(x_{j+1},y_{j+1}) = T_i(x_j,y_j)$: j = 0, 1, 2, …

generated by iterating T_i repeats itself.

If all of your test case sequences (x_i,y_i) above are finite, does there exist a metamorphic equation *e* for `SetEquality` with a transformer *T* which can generate a test suite of TS = { (x_0,y_0), …, $(x_k,y_k)= T^k(x_0,y_0)$ } of unlimited size by simply iterating the transformer *T k*-times?

Model Answer

(1) For $T_1(l_1,l_2)) = (l_2,l_1)$ we see that $T_1(T_1(l_1,l_2))) = (l_1,l_2)$. So, iteration of T_1 only gives a test suite of size 2.

(2) For $T_2(l_1,l_2)) = (l_1,\text{reverse}(l_2))$ we see that $T_2(T_2(l_1,l_2))) = (l_1,l_2)$. So, iteration of T_2 only gives a test suite of size 2.

(3) For $T_3(l_1,l_2)) = (l_1,\text{permute}(l_2))$ we see that the number of repeated permutations $\text{permute}^n(l_2)$ is finite, and some subset of all possible permutations of l_2. So the sequence $(x_{j+1},y_{j+1}) = T_i(x_j,y_j) : j = 0, 1, 2, \ldots$ must repeat itself, and iteration of T_3 gives a finite (but possibly large if l_2 is long) test suite

(4) For $T_4\ (l_1,l_2)) = (\text{append}(l_1, c),\text{append}(\text{append}(l_2,c),c))$ the sequence

$$(x_{j+1},y_{j+1}) = T_4(x_j,y_j) : j = 0, 1, 2, \ldots$$

never repeats itself, as both x_i and y_i grow arbitrarily long as lists. So, the test suite

$$TS = \{ (x_0,y_0), \ldots, (x_k,y_k)= T^k(x_0,y_0) \}$$

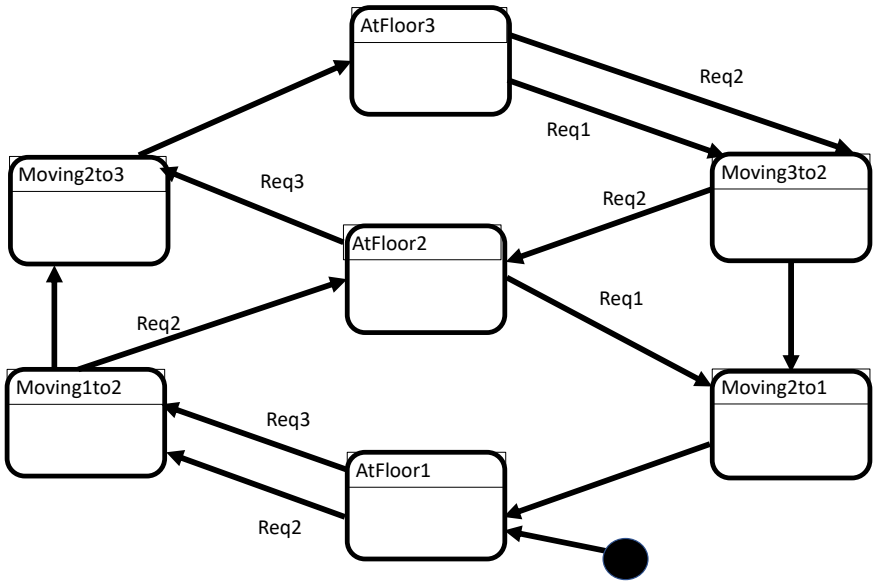
given by repeated iterations of the transformer T_4 can be defined for any desired size k.

(5) For some suitable choices of injective f: int -> int iteration of T_5 will generate arbitrarily many new test cases, e.g. f(x) = x+1, f(x) = x-1 but not for all choices of injective f, e.g. f(x)=x or f(x) = -x.

Question 3. (Total 35 points)

A building has 3 floors which are serviced by an elevator connecting all floors. An elevator passenger must request that the elevator car comes to her/his floor by means of a request button next to the elevator door on each floor. The three request buttons on floors 1, 2 and 3, when pressed, generate the system events *Req1*, *Req2* and *Req3* respectively. **Note: to simplify this question we will ignore any buttons inside the elevator itself.**

Michael Modelmad drew the following UML Statechart, as a model to design and test the elevator software.



(i) **(2 points)** Define all 7 different trap properties, using LTL, that would give 100% node coverage of Michael's model.

Model Answer
We can introduce a state variable, and we get 7 node trap properties, one for each state:
G(!State == AtFloor1), G(!State == AtFloor2), G(!State == AtFloor3)
G(!State == Moving1to2), G(!State == Moving2to3), G(!State == Moving3to2),
G(!State == Moving2to1)

(ii) **(3 points)** Define the smallest number of LTL node trap properties that would also give 100% node coverage of Michael's model.

Model Answer
Clearly some trap properties imply others, so the smallest set of node trap properties is three:
G(!State == Moving3to2), G(!State == AtFloor2), G(!State == Moving2to1)

(iii) **(6 points)** Describe using natural language (English or Swedish) the requirements for correct functional behaviour of the elevator from the perspective of an end user standing on floor 1, floor 2 and floor 3 respectively.
Model Answer
An elevator is not a hard real-time system, sometimes to the users frustration. So the appropriate temporal requirements are:

Floor 1 requirement: Whenever the user presses the request button on floor 1 then eventually the elevator arrives at floor 1.
Floor 2 requirement: Whenever the user presses the request button on floor 2 then eventually the elevator arrives at floor 2.

Floor 3 requirement: Whenever the user presses the request button on floor 3 then eventually the elevator arrives at floor 3.

(iv) **(6 points)** Model your 3 answers to part (iii) above as 3 separate LTL formulas. Each formula should precisely capture the meaning of the corresponding natural language requirement. **Hint**: you may use a *State* variable in your LTL formulas.

Model Answer

G(Req1 => F(State == AtFloor1)
G(Req2 => F(State == AtFloor2)
G(Req3 => F(State == AtFloor3)

(v) **(6 points)** Michael's sister Mary said that the Michael's model above was wrong. She was able to define three counter-examples, (i.e. execution sequences of the model) which demonstrated that Michael's model violates all of its three functional behaviour requirements. Write down three counterexamples that Mary could have defined.

Model Answer

The formulas in part (iv) are classical LTL liveness formulas. The only possible counterexamples to such formulas are infinite sequences of inputs. Such infinite sequences must be written down using some sort of loop construction, as indicated below.

(i) **The infinitely repeating event sequence**
Req2, Req3, Req3, Req1, Req2, Req3, Req3, Req1, Req2, Req3, Req3, …
Traverses the states
AtFloor1, Moving1to2, Moving2to3, AtFloor3, Moving3to2, AtFloor2, Moving2to3, AtFloor3, Moving3to2, AtFloor2, Moving2to3, AtFloor3, …
In this case floor 1 is actually requested infinitely often and not just once. But apart from during the initial state, the elevator never visits floor 1.

(ii) **The infinitely repeating event sequence**
Req2, Req3, Req3, Req2, Req3, Req3,…
leads to the infinitely repeating state sequence
AtFloor1, Moving1to2, Moving2to3, AtFloor3, Moving3to2, Moving2to1, AtFloor1,
In this case floor 2 is again requested infinitely often, but the elevator never arrives at floor 2.

(iii) **The infinitely repeating event sequence**
Req3, Req2, Req1, Req3, Req3, Req2, Req1, …
leads to the infinitely repeating state sequence
AtFloor1, Moving1to2, AtFloor2, Moving2to1,AtFloor1, Moving1to2, AtFloor2, Moving2to1, …
In this case floor 3 is again requested infinitely often, but the elevator never arrives at floor 3.

(vi) **(2 points)** Comment on whether your counterexamples in part (v) could be used as test cases for an implementation of the elevator software. Is it possible to test the elevator software for correctness? Motivate your answer.

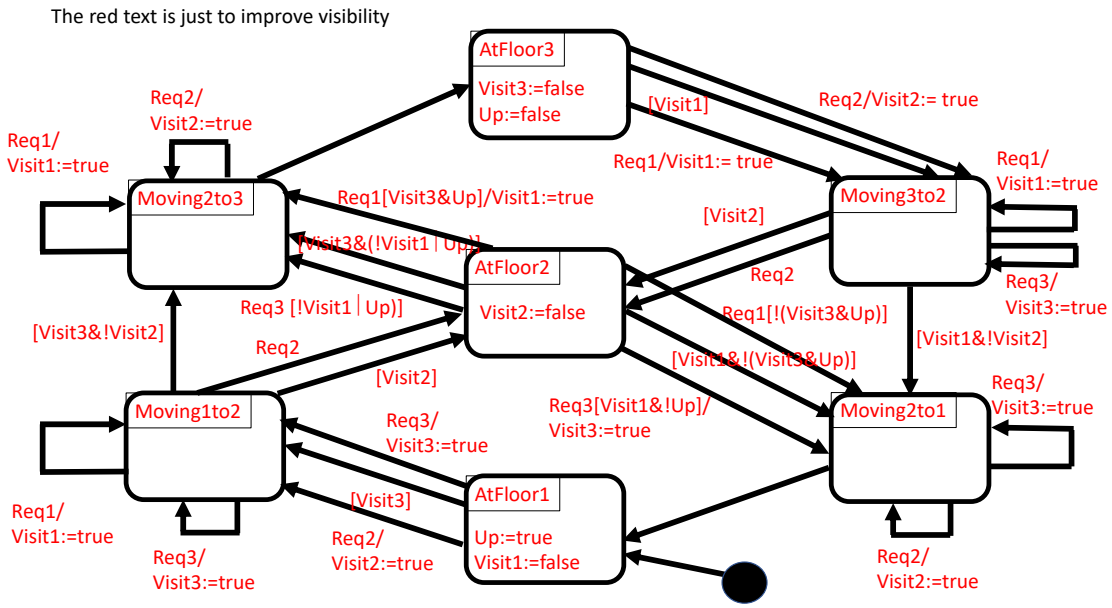
Model Answer

The counterexamples in part (v) are (and must be) infinitely long repeating sequences (inputs) which could not therefore be used as test cases in finite time.

From the structure of the LTL formulas in part (iv) we can see that no finite test case could ever show that an elevator implementation is incorrect. There is always the possibility that after any floor request ReqX the lift would arrive at a floor X eventually. As mentioned before, the correctness formulas of part (iv) are typical examples of liveness formulas which are not amenable to testing.

(vii) **(10 points)** By introducing <u>just 4 new Boolean variables</u>, **but no new states**, show how you can <u>correct</u> Michael's model to a functionally correct UML Statechart model of an elevator with respect to the user requirements of Part (iv). Use your execution sequences from Part (v) to check if your corrected design fails any Part (iv) requirement. **Hint:** you <u>may</u> change existing transitions and/or introduce new transitions as well as changing conditions and adding assignments for the 4 variables you introduce.

Model Answer



Notice that the three counterexamples of Part (v) above now execute correctly, i.e. if a

floor button is pressed the floor request is met. The elevator system is now fair.

(Please turn over)

Question 4. (Total 20 points)

(i) **(6 points)** Recall that a test case *tc* for a condensation graph *G* of a program *P satisfies* a control flow test requirement $p = v_1, \ldots, v_n$ on *G* if and only if executing *P* on *tc* causes *P* to take the path *p* some time during its execution.

We can define the relationship "*implies*" between any two control flow test requirements *reqA* and *reqB* on a condensation graph *G*, by *reqA* implies *reqB* if and only if for every test case *tc* for *G*,

if *tc* satisfies *reqA* on *G* then *tc* satisfies *reqB* on *G*.

If *reqA* implies *reqB* then in symbols we will write ***reqA ⇒ reqB***, and if both *reqA* ⇒ *reqB* and reqB ⇒ reqA then we will write **reqA ⇔ reqB**.

Give rigorous and sound arguments (i.e. proofs or counterexamples) to show (a), (b) and (c) below:

(a) If *reqA* ⇒ *reqB* and *reqB* ⇒ *reqC* then *reqA* ⇒ *reqC*

Model Answer
Consider any test case *tc* for *G* and suppose *tc* satisfies *reqA*. We must show *tc* satisfies *reqC.* Now since *reqA* ⇒ *reqB* then *tc* satisfies *reqA.* Then since *reqB* ⇒ *reqC tc* satisfies *reqC* and we have proven the result.

(b) If *reqA* ⇒ *reqB* it does not necessarily follow that *reqB* ⇒ *reqA.*

Model Answer
Any branching path such as

```
A; if (cond) then B else C
```

has the property that *reqB* ⇒ *reqA* but not *reqA* ⇒ *reqB* unless cond is tautologically true.

(c) If *reqA* ⇔ *reqB* then tc satisfies *reqA* if and only if tc satisfies *reqB* and hence *reqA* and *reqB* have exactly the same satisfying test cases.

Model Answer

Suppose that *reqA* ⇔ *reqB.* Then consider any test case *tc* for *G.* If tc satisfies *reqA* then by assumption *reqA* ⇒ *reqB* so tc satisfies *reqB.* Conversely, if tc satisfies *reqB* then by assumption *reqB* ⇒ *reqA* so tc satisfies *reqA.*

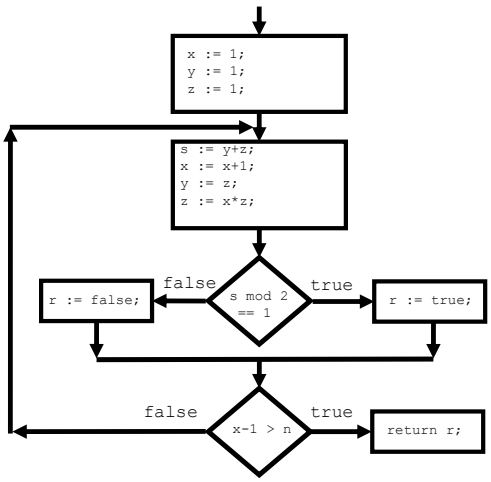
(ii) **(total 14 points**) Harry Hopeful wanted to unit test the following code

```
1. myTest(int n): boolean
2. {
3.       int x, y, z, s, r;

4.       x:= 1;
5.       y:= 1;
6.       z:= 1;
7.       repeat {
8.            s:= y+z;
9.            x:= x+1;
10.           y:= z;
11.           z:= x*z;
12.           if (s mod 2 == 1) then
13.                r:=true
14.           else
15.                r:= false;
16.      } until (x-1 > n);
17.      return r;
18.      }
```

(a) (2 points) Draw a condensation graph for Harry's code.

Model Answer



(b) (2 points) Harry wrote the following JML precondition for his code

```
requires n >= 0
```

Write down the correct JML postcondition for this precondition, based on an analysis of the behaviour of Harry's code.

Model Answer
The code produces a result true if, and only if n! + (n+1)! is odd. So from the code

ensures
((\product int i; 1 <= i <= \old(n); i) + (\product int i; 1 <= i <= \old(n+1); i))
mod 1 == 1 <=>
\result == true

And by analysis of this postcondition, we can simplify to
ensures
\result == true <=> \old(n) == 1

(c) (10 points) Harry thought that the code on line 13 was dead code so that he could never achieve 100% line coverage in testing. Using the method of symbolic evaluation write a constraint table which shows that Harry was wrong. Write out the test case derived from your constraint table and use your postcondition from part (b) to predict your test case output.

Model Answer
For line coverage, we can analyse the code line-by-line rather than nodewise. Below, we analyse a complete terminating path:

Line#	Instruction	Constraint
4	`x := 1`	x= 1
5	`y := 1`	y = 1
6	`z := 1`	z = 1
8	`s := y + z`	s =2
9	`x:= x+1`	x=2
10	`y:=z`	y=1
11	`z:=x*z`	z=2
12	`s mod 2 ==1`	false
15	`r:=false`	r = false
16	`!(x-1>n)`	n≥1
8	`s:=y+z`	s=3
9	`x:=x+1`	x=3
10	`y:=z`	y=2
11	`z:=x*z`	z=6
12	`s mod 2 == 1`	true
13	`r:=true`	r=true

16	`x-1>n`	**$n \geq 1$ & $n < 2 \equiv n = 1$**
17	`return r`	

From line #16 in the table above, a test case taking this terminating path is constrained to be n=1, in which case line 13 is executed and therefore not dead code.

Using our postcondition from part (b) above since n = 1 then the predicted output is `true`.